Scope, Related Work, and Positioning

Scope, Related Work, and Positioning I

This section situates the exemplar and states explicit boundaries. The goal is not to compete with BPL's substantially larger full compiler pipeline (Bota Biosciences 2026) — with its Lark grammar, robot backend, and hash-chained audit/compliance layer — but to show how a minimal, test-backed subset of BPL's domain-language design fits the template's reproducibility and rendering stack (Peng 2011), generalized from wet-lab protocols to any controlled procedure.

Domain-specific languages for controlled procedures I

Encoding a procedure as a program rather than free text is a long-standing software-engineering pattern: Fowler's treatment of domain-specific languages (Fowler 2010) frames the general case — a notation restricted to exactly the concepts a domain needs. BPL (Bota Biosciences 2026) applies this specifically to laboratory protocols, adding biology-native types (reagents, labware, MW-aware concentrations), staged validation, and deterministic compilation to a robot or human execution target. The present manuscript restricts attention to the parts of that design that generalize beyond biology: a controlled vocabulary of step intents, a small dimensional-safety unit system, staged validation gates, and deterministic compilation to a hashed plan.

What this exemplar does not implement (out of scope) I

What this exemplar does not implement (out of scope) I

1. **A text grammar and parser.** BPL parses `.bpl` source through a Lark grammar into a typed AST. This exemplar constructs a `Method` directly as frozen Python dataclasses — no concrete syntax, no parser, no AST layer.
2. **Biology-native types.** BPL's unit system includes MW-aware concentration conversions and reagent physical-form metadata (`cas`, `physical_form`). This exemplar's `units.py` implements only the dimensional-safety subset (mass, volume, temperature, time, molar concentration, mass concentration, count, dimensionless) needed to demonstrate the "the type system catches `mL + g`" guarantee — molar and mass concentration are kept as distinct dimensions precisely because no MW-aware conversion exists here to safely mix them.
3. **A robot backend.** BPL lowers intents to Biomek i7 primitives (`aspirate`, `dispense`, `pick_tips`). This exemplar's `Target.AUTOMATED` has no backend-specific lowering stage; it is a scheduling and gate-compatibility concept only.
4. **Cryptographic audit guarantees.** `trust.py`'s hash-chain is deliberately scoped to the same honest boundary BPL itself claims: a consistency check against accidental corruption, not a tamper-proof guarantee against an actor with write access to the entire chain.

What this exemplar does not implement (out of scope) II

5. **SOP-to-DSL generation.** BPL includes an agentic workflow that translates natural-language SOPs into validated `.bpl` programs. This exemplar's worked examples are hand-authored Python, not generated.

What this project proves about the template I

The validation and compilation steps here are a deliberately small subset of BPL's. The **non-standard** contribution is procedural: the same tested functions in `src/methods_dsl/` back the analysis script, the test suite, and this manuscript, so the compiled-plan table and the figure always refer to the same code. That pattern — and the specific generalization from a biology-only domain language to a domain-neutral one — is what downstream projects should copy, whether the controlled procedure is a wet-lab protocol, an instrument calibration sweep, or a computational pipeline.

Explicit limitations I

Explicit limitations I

1. **Two worked examples**: `PBSPreparation` and `SensorCalibrationSwee p` exercise every gate-success path but are not a corpus; gate-failure coverage instead lives in `tests/conftest.py`'s dedicated fixtures.
2. **No backend lowering**: `Target` selects a compatibility class, not a concrete execution backend; no robot or simulation runtime exists in this exemplar.
3. **Unit table, not a full unit library**: eight dimensions and a fixed unit table, not a general-purpose dimensional-analysis library like `pint`.
4. **No persistence layer**: `trust.py`'s hash-chain lives in memory for the duration of one script run; this exemplar does not implement durable storage for a real audit trail.

Explicit limitations I

These limitations are intentional: they narrow the surface so that the reproducibility concerns — tested functions, a thin script, and generated-variable-injected prose — remain visible rather than buried under a compiler implementation at BPL's full scale.